Universidad del Valle de Guatemala
Facultad de Ingeniería
POO
Segundo Ciclo, 2026
Sección # 21



Laboratorio 3. Herencia

Antonio Molina - 26878

Requisitos Funcionales

- El sistema debe permitir registrar órdenes de servicio técnico a domicilio.
- Cada orden debe guardar un código único, nombre del cliente, dirección, horas estimadas, zona, estado y cantidad de visitas realizadas.
- No deben existir dos órdenes con el mismo código.
- Las horas estimadas pueden contener decimales, pero deben ser mayores que cero.
- El sistema debe manejar dos zonas: metropolitana y periférica.
- El sistema debe manejar cuatro categorías de servicio: diagnóstico, instalación, instalación eléctrica certificada y mantenimiento de aire acondicionado.
- Los diagnósticos no pueden tener más de 1.5 horas estimadas.
- Las instalaciones deben guardar la cantidad de puntos a instalar, que debe ser un entero positivo.
- Las instalaciones eléctricas certificadas deben guardar el valor declarado de los materiales, necesario para calcular la garantía.
- Los mantenimientos de aire acondicionado deben guardar la capacidad del equipo en BTU, entre 9,000 y 60,000.
- Los mantenimientos de aire acondicionado solamente pueden realizarse en la zona metropolitana.
- Cada cuadrilla debe guardar un código único, su capacidad máxima de horas por jornada y si está certificada en refrigeración.
- El sistema debe guardar las jornadas y las órdenes asignadas a cada una.
- El sistema debe registrar por separado los ingresos de órdenes confirmadas y los ingresos por visitas adicionales.
- El costo de un diagnóstico no debe cambiar por las horas estimadas.
- En una instalación, las horas por puntos se calculan multiplicando la cantidad de puntos por 0.75.
- Las horas facturables de una instalación deben ser el mayor valor entre las horas estimadas y las horas por puntos.
- Las horas facturables deben redondearse hacia arriba al entero siguiente cuando exista una fracción.
- Una instalación eléctrica certificada debe incluir el costo normal de instalación, el certificado y la garantía.
- Los cargos por visitas adicionales deben ser de Q40 por cada visita posterior a la primera, registrados cuando inicia la jornada.
-

Reglas de cobro y tiempos de atención

- Diagnóstico Q150 metropolitana / Q225 periférica 1 / 2 días
- Instalación Base Q100 / Q175 + Q90 por hora facturable o fracción 3 / 5 días
- Instalación eléctrica certificada Costo de instalación + Q120 + 5 % de materiales 5 / 7 días
- Aire acondicionado Base por zona + Q110 por hora estimada + Q80 de gas 2 días

Operaciones y condiciones

- El sistema debe permitir cotizar servicios sin registrarlos ni modificar los ingresos.
- aceptar una cotización, debe registrarse la orden, cobrar el monto completo y dejarla pendiente.
- El sistema debe permitir registrar cuadrillas y evitar códigos repetidos.
- El sistema debe permitir crear jornadas para las cuadrillas.
- Una cuadrilla no puede tener más de una jornada sin cerrar.
- .El sistema debe permitir asignar órdenes pendientes a una jornada en preparación.
- La suma de horas estimadas asignadas no puede superar la capacidad de la cuadrilla.
- Los mantenimientos de aire acondicionado solo pueden asignarse a cuadrillas certificadas en refrigeración.
- Una orden no puede estar asignada a dos jornadas al mismo tiempo.
- El sistema debe permitir retirar órdenes de una jornada en preparación, devolviéndolas al estado pendiente.
- No se puede iniciar una jornada vacía.
- Al iniciar una jornada, sus órdenes deben pasar al estado en atención.
- El sistema debe permitir registrar el resultado de cada visita como completada o fallida.
- Una visita fallida debe devolver la orden a pendiente si todavía tiene visitas disponibles.
- Cuando una orden agota sus visitas permitidas, debe quedar cancelada.
- Los diagnósticos e instalaciones normales permiten hasta 3 visitas.
- Las instalaciones eléctricas certificadas permiten hasta 2 visitas.
- Los mantenimientos de aire acondicionado permiten solamente 1 visita.
- La jornada debe cerrarse cuando todas sus órdenes tengan un resultado registrado.
- Cuando una jornada se cierra, la cuadrilla debe quedar disponible.
- El sistema debe permitir consultar órdenes, cuadrillas y jornadas.
- El reporte debe mostrar la cantidad de órdenes por categoría y estado.
- El reporte debe mostrar los ingresos por registro, por visitas adicionales y por categoría.
- El reporte debe mostrar las jornadas sin cerrar, sus cuadrillas, horas asignadas y capacidad disponible.
- Las cotizaciones, asignaciones retiradas y operaciones rechazadas no deben modificar los ingresos.
- El programa debe comenzar con al menos dos órdenes de cada categoría y tres cuadrillas.
- El programa debe manejar entradas incorrectas sin terminar inesperadamente.
- El menú debe permanecer disponible hasta que el usuario seleccione salir.

Decisiones que dependen de la categoría

Decisión	Clase responsable
Costo fijo y máximo de 1.5 horas	Diagnostica
Cálculo de horas facturables y costo de instalación	Instalación
Certificado y garantía de materiales	InstalacionElectrica
Validación de BTU y cargo por refrigerante	MantenimientoAire
Tiempo estimado de atención	Cada categoría
Cantidad máxima de visitas	Cada categoría
Compatibilidad con cuadrilla	Cada categoría

La clase padre OrdenServicio tendrá métodos comunes como calcularCosto(), calcularTiempoAtencion(), maximoVisitas() y esCompatible(Cuadrilla cuadrilla).
Cada clase hija implementará estos métodos según sus reglas. De esta forma, cuando el sistema trabaje con una orden, podrá llamar directamente a sus métodos sin tener que revisar su categoría.

Estados de las órdenes y jornada

Estados en una orden

Los estados que utilizaremos serán Pendiente, Asignada, En atención, Completada y Cancelada.
Una orden inicia pendiente cuando se registra. Al agregarla a una jornada pasa a asignada y, cuando la jornada inicia, pasa a atención. Si se completa el servicio, termina como completada. Si falla la visita, vuelve a pendiente cuando todavía puede reprogramarse; si agotó sus visitas, queda cancelada.

Estados de una jornada

Empieza en preparación, luego pasa a iniciada, Cerrado
Mientras está en preparación se pueden asignar y retirar órdenes. Después de iniciarla ya no se permiten esos cambios. Al registrar todos los resultados, la jornada queda cerrada y libera a la cuadrilla.

Clases necesarias y sus responsabilidades

Clase OrdenServicio

La clase OrdenServicio representa la información y los comportamientos que tienen en común todas las órdenes de servicio de TecniHogar. Se encargará de guardar el código de la orden, nombre del cliente, dirección, horas estimadas, zona, estado y cantidad de visitas realizadas. Esta clase será la clase padre de Diagnostico, Instalacion y MantenimientoAire. También será la clase antecesora de InstalacionElectrica, que heredará directamente de Instalacion. La clase OrdenServicio será abstracta porque no se necesita crear una orden

sin una categoría específica. También tendrá métodos comunes para administrar los estados de las órdenes y métodos abstractos para calcular el costo, el tiempo estimado de atención y la cantidad máxima de visitas.

Clase Diagnostico
La clase Diagnostico hereda de OrdenServicio y representa los servicios que se realizan para revisar un problema antes de repararlo. Esta clase se encargará de validar que las horas estimadas no sean mayores que 1.5 y de calcular el costo fijo dependiendo de la zona. Si el servicio se realiza en la zona metropolitana, su costo será de Q150, mientras que en la zona periférica será de Q225. También determinará el tiempo estimado de atención, que será de un día para la zona metropolitana y dos días para la periférica. Esta categoría permitirá un máximo de tres visitas.

Clase Instalacion
La clase Instalacion hereda de OrdenServicio y representa los servicios en los que se deben instalar diferentes puntos. Además de los datos comunes, guardará la cantidad de puntos a instalar, que debe ser un número entero mayor que cero. Esta clase se encargará de calcular las horas facturables utilizando el mayor valor entre las horas estimadas y las horas calculadas por los puntos. Después calculará el costo utilizando la tarifa base de la zona y agregando Q90 por cada hora facturable o fracción. También determinará el tiempo estimado de atención, que será de tres días para la zona metropolitana y cinco para la periférica. Las instalaciones normales permitirán un máximo de tres visitas.

Clase InstalacionElectrica
La clase InstalacionElectrica hereda de Instalacion porque representa un servicio de instalación que necesita características adicionales. Además de la información heredada, guardará el valor declarado de los materiales utilizados. Esta clase utilizará el cálculo de costo de una instalación normal y agregará Q120 por el certificado y un 5 % del valor de los materiales por la garantía. También tendrá un tiempo estimado de atención de dos días adicionales al de una instalación normal. A diferencia de las instalaciones normales, permitirá un máximo de dos visitas.

Clase MantenimientoAire
La clase MantenimientoAire hereda de OrdenServicio y representa los servicios de mantenimiento de equipos de aire acondicionado. Además de los atributos comunes, guardará la capacidad del equipo en BTU. Esta clase se encargará de validar que la capacidad esté entre 9,000 y 60,000 BTU y que el servicio solamente se registre en la zona metropolitana. También calculará el costo del mantenimiento utilizando la tarifa base correspondiente, Q110 por cada hora estimada y Q80 adicionales por el gas refrigerante. Su tiempo estimado de atención será siempre de dos días y permitirá solamente una visita. Además, esta clase será responsable de indicar que sus órdenes solamente pueden asignarse a cuadrillas certificadas en refrigeración.

Clase Cuadrilla
La clase Cuadrilla representa a los grupos de técnicos que realizan los servicios a domicilio. Se encargará de guardar el código que identifica a la cuadrilla, la capacidad máxima de horas que puede trabajar durante una jornada y si cuenta con certificación en refrigeración. Esta información se utilizará para verificar si una cuadrilla puede atender una orden y si

tiene suficiente capacidad para realizar los servicios asignados. También permitirá consultar los datos de la cuadrilla para mostrarlos en el menú y en los reportes. La clase no calculará los costos de las órdenes, porque esa responsabilidad corresponde a las categorías de servicio.

Clase Jornada

La clase Jornada representa un conjunto de órdenes de servicio asignadas a una cuadrilla durante su jornada de trabajo. Se encargará de guardar la cuadrilla responsable, las órdenes asignadas y el estado actual de la jornada. También permitirá agregar y retirar órdenes mientras la jornada se encuentre en preparación. Antes de asignar una orden, verificará que esté pendiente, que no esté asignada a otra jornada, que la cuadrilla sea compatible y que las horas estimadas no superen su capacidad. Cuando la jornada inicie, cambiará las órdenes al estado en atención y registrará las visitas correspondientes. También permitirá registrar si cada visita fue completada o fallida. La jornada quedará cerrada cuando todas sus órdenes tengan un resultado registrado. Esto permitirá que la cuadrilla pueda utilizarse nuevamente.

Clase TecniHogar

La clase TecniHogar representa a la empresa y se encargará de administrar la información general del sistema. Guardará las órdenes registradas, las cuadrillas disponibles y las jornadas creadas. También se encargará de registrar nuevas órdenes y cuadrillas, evitar códigos repetidos, buscar información y coordinar las operaciones relacionadas con las jornadas. Esta clase administrará los ingresos de la empresa, separando el dinero recibido por el registro de órdenes y el dinero obtenido por visitas adicionales. Los ingresos por registro solamente aumentarán cuando un cliente acepte una cotización y confirme su orden. Los cargos por visitas adicionales se registrarán cuando comience la jornada correspondiente, no cuando se asigne una orden. También proporcionará la información necesaria para generar los reportes, incluyendo las órdenes por categoría y estado, los ingresos y las jornadas que todavía no se encuentran cerradas. De esta manera, la administración de la empresa no estará directamente dentro de Main.

Clase Main

La clase Main será el programa principal y contendrá el método main(). Se encargará de crear el objeto TecniHogar, cargar los datos iniciales y mostrar el menú de opciones. También solicitará al usuario los datos necesarios para cotizar y registrar órdenes, registrar cuadrillas, crear jornadas, asignar y retirar órdenes, iniciar jornadas, registrar resultados y consultar reportes. Esta clase utilizará los métodos de las demás clases para realizar las operaciones, evitando colocar directamente los cálculos de cada categoría dentro del menú. También manejará las entradas incorrectas para que el programa no termine inesperadamente.

Atributos, tipos, visibilidad y propósito

Todos los atributos serán privados (private) para mantener el encapsulamiento. De esta manera, los datos de los objetos no podrán modificarse directamente desde otras clases y se utilizarán métodos para acceder a ellos cuando sea necesario.

Clase OrdenServicio

- codigo (String): guarda el código que identifica a cada orden de servicio. Se utiliza para buscar una orden y evitar que existan dos órdenes con el mismo código.
- cliente (String): guarda el nombre del cliente que solicita el servicio técnico.
- direccion (String): guarda la dirección donde se realizará el servicio a domicilio.
- horasEstimadas (double): guarda la cantidad de horas que se estima que tomará realizar el trabajo. Se utiliza para calcular algunos costos y verificar la capacidad de las cuadrillas.
- zona (String): indica si el servicio se realizará en la zona metropolitana o periférica. Este dato se utiliza para determinar el costo y el tiempo estimado de atención.
- estado (String): guarda el estado actual de la orden. Puede ser Pendiente, Asignada, En atención, Completada o Cancelada.
- visitasRealizadas (int): guarda la cantidad de visitas que se han realizado para atender la orden. Se utiliza para determinar si todavía puede reprogramarse o si debe cancelarse.

Clase Diagnostico

Esta clase no necesita atributos adicionales porque toda la información que utiliza se encuentra en OrdenServicio.

El costo, tiempo estimado y cantidad máxima de visitas se determinarán mediante sus propios métodos.

Clase Instalacion

- cantidadPuntos (int): guarda la cantidad de puntos que deben instalarse durante el servicio. Debe ser un número entero mayor que cero y se utiliza para calcular las horas facturables.

Clase InstalacionElectrica

- valorMateriales (double): guarda el valor declarado de los materiales utilizados durante la instalación eléctrica certificada. Se utiliza para calcular el 5 % adicional correspondiente a la garantía.

Los demás atributos necesarios se heredan de Instalacion y OrdenServicio.

Clase MantenimientoAire

- capacidadBTU (int): guarda la capacidad del equipo de aire acondicionado. Debe encontrarse entre 9,000 y 60,000 BTU.

Este atributo se utiliza para validar que el equipo pueda recibir el servicio.

Clase Cuadrilla

- codigo (String): guarda el código que identifica a la cuadrilla. Se utiliza para buscarla y evitar códigos repetidos.
- capacidadHoras (double): guarda la cantidad máxima de horas que la cuadrilla puede trabajar durante una jornada. Debe ser mayor que cero.

- certificadaRefrigeracion (boolean): indica si la cuadrilla cuenta con certificación para realizar mantenimientos de aire acondicionado. Se utiliza para verificar si puede atender ese tipo de orden.

Clase Jornada

- codigo (String): guarda el código que identifica a la jornada y permite buscarla dentro del sistema.
- cuadrilla (Cuadrilla): guarda la referencia a la cuadrilla encargada de realizar los servicios asignados.
- ordenes (ArrayList de OrdenServicio): almacena las órdenes que se encuentran asignadas a la jornada. Permite agregar y retirar órdenes mientras la jornada está en preparación.
- estado (String): guarda el estado actual de la jornada. Puede ser Preparación, Iniciada o Cerrada.
- resultadosRegistrados (ArrayList de Boolean): guarda qué órdenes ya tienen un resultado de visita registrado. Cada posición corresponde a una orden de la colección ordenes. Se utiliza para saber cuándo todas las visitas han sido registradas y se puede cerrar la jornada.

Clase TecniHogar

- ordenes (ArrayList de OrdenServicio): almacena todas las órdenes registradas en la empresa. Puede contener objetos de las diferentes categorías gracias a la herencia.
- cuadrillas (ArrayList de Cuadrilla): guarda las cuadrillas registradas y permite buscarlas para crear jornadas.
- jornadas (ArrayList de Jornada): almacena las jornadas creadas y permite consultar cuáles se encuentran en preparación, iniciadas o cerradas.
- ingresosRegistro (double): guarda el dinero acumulado por las órdenes que han sido confirmadas y registradas. No aumenta cuando solamente se realiza una cotización.
- ingresosVisitas (double): guarda el dinero acumulado por las visitas adicionales. Cada visita posterior a la primera genera Q40, que se cobran cuando inicia la jornada correspondiente
- categoriasIngresos (ArrayList de String): almacena los nombres de las categorías que han generado ingresos durante la ejecución del programa. Permite identificar cada categoría sin tener que verificar el tipo específico de las órdenes.
- montosCategorias (ArrayList de Double): almacena el dinero acumulado por cada categoría. Cada posición corresponde a la misma posición de categoriasIngresos, permitiendo mantener relacionados los nombres y los montos.

.

Clase Main

- teclado (Scanner): atributo privado y estático que permite leer los datos y las opciones ingresadas por el usuario.
- empresa (TecniHogar): atributo privado y estático que guarda el objeto encargado de administrar las órdenes, cuadrillas, jornadas e ingresos.

Valores iniciales de los atributos

Cuando se crea una orden de servicio, su código, cliente, dirección, horas estimadas y zona se reciben mediante el constructor. El estado de una orden comenzará como Pendiente y su cantidad de visitas realizadas comenzará en 0.
Los atributos particulares de cada categoría se inicializarán mediante sus constructores. Por ejemplo, Instalacion recibirá la cantidad de puntos, InstalacionElectrica recibirá el valor de los materiales y MantenimientoAire recibirá la capacidad en BTU.

- Cuando se crea una cuadrilla, su código, capacidad máxima y certificación se reciben mediante el constructor.

- Cuando se crea una jornada, su estado comenzará como Preparación y su colección de órdenes estará vacía. La colección que controla los resultados también comenzará vacía.

- Cuando se crea TecniHogar, las colecciones de órdenes, cuadrillas y jornadas comenzarán vacías. Los ingresos por registro y por visitas adicionales comenzarán en 0.

Después, desde Main, se registrarán los datos iniciales que exige el ejercicio: al menos dos órdenes pendientes de cada categoría y tres cuadrillas, incluyendo una certificada y otra no certificada en refrigeración. Estas órdenes iniciales no deben generar ingresos.

Métodos, parámetros, tipos de retorno y propósito

Los métodos serán públicos cuando necesiten utilizarse desde otras clases. Los métodos auxiliares podrán ser privados. Los constructores permitirán inicializar los objetos y los métodos que modifican estados verificarán que las operaciones sean válidas antes de realizar cambios.

Clase OrdenServicio

- OrdenServicio(String codigo, String cliente, String direccion, double horasEstimadas, String zona): constructor protegido que recibe los datos comunes de una orden. Valida que el código no esté vacío, que las horas sean mayores que cero y que la zona sea metropolitana o periférica. Inicializa el estado como Pendiente y las visitas realizadas en cero. Las clases hijas lo utilizarán mediante super().
- getCodigo() : String: devuelve el código que identifica la orden.
- getCliente() : String: devuelve el nombre del cliente que solicitó el servicio.
- getDireccion() : String: devuelve la dirección donde se realizará el trabajo.
- getHorasEstimadas() : double: devuelve las horas estimadas del servicio. Se utiliza para comprobar la capacidad disponible de una cuadrilla.
- getZona() : String: devuelve la zona en la que se realizará el servicio.
- getEstado() : String: devuelve el estado actual de la orden.
- getVisitasRealizadas() : int: devuelve la cantidad de visitas realizadas para atender la orden.
- calcularCosto() : double: método público y abstracto que devuelve el costo del servicio. Cada clase hija implementará su propia forma de calcularlo.
- calcularTiempoAtencion() : int: método público y abstracto que devuelve la cantidad de días hábiles estimados para atender la orden, dependiendo de su categoría y zona.

- maximoVisitas() : int: método público y abstracto que devuelve la cantidad máxima de visitas permitidas para la categoría.
- getCategoria() : String: método público que devuelve el nombre de la categoría de la orden. Cada clase hija tendrá su propia implementación para que los reportes puedan identificar las categorías sin utilizar instanceof.
- esCompatible(Cuadrilla cuadrilla) : boolean: método público que recibe una cuadrilla y devuelve si puede atender la orden. En la clase padre permitirá cualquier cuadrilla válida, mientras que MantenimientoAire sobrescribirá el método para exigir certificación en refrigeración.

- signar() : boolean: cambia el estado de Pendiente a Asignada. Devuelve true cuando se realiza correctamente y false si la orden no puede asignarse.
- retirar() : boolean: devuelve una orden Asignada al estado Pendiente. No modifica las visitas realizadas ni los ingresos.
- iniciarAtencion() : boolean: cambia el estado de Asignada a En atención y aumenta la cantidad de visitas realizadas. Devuelve true si la operación se realiza correctamente.
- registrarResultado(boolean completada) : boolean: registra el resultado de una visita. Si fue completada, cambia el estado a Completada. Si fue fallida y todavía quedan visitas disponibles, vuelve a Pendiente. Si agotó sus visitas, cambia a Cancelada.
- toString() : String: devuelve una cadena con los datos generales de la orden para mostrarlos en las consultas.

Clase Diagnostico

- Diagnostico(String codigo, String cliente, String direccion, double horasEstimadas, String zona): constructor que recibe los datos de la orden y utiliza super() para inicializar los atributos comunes. También valida que las horas estimadas no superen 1.5.
- calcularCosto() : double: devuelve Q150 si la zona es metropolitana o Q225 si es periférica. No utiliza las horas estimadas para calcular el costo.
- calcularTiempoAtencion() : int: devuelve 1 día para la zona metropolitana y 2 días para la periférica.
- maximoVisitas() : int: devuelve 3 porque los diagnósticos permiten hasta tres visitas.
- getCategoria() : String: devuelve el texto "Diagnostico" para identificar la categoría en los reportes.

Clase Instalacion

- Instalacion(String codigo, String cliente, String direccion, double horasEstimadas, String zona, int cantidadPuntos): constructor que inicializa los datos comunes mediante super() y guarda la cantidad de puntos. Valida que la cantidad sea mayor que cero.
- getCantidadPuntos() : int: devuelve la cantidad de puntos que deben instalarse.
- calcularHorasFacturables() : int: calcula las horas por puntos multiplicando la cantidad de puntos por 0.75. Después compara ese resultado con las horas estimadas, utiliza el mayor valor y lo redondea hacia arriba para obtener las horas que se cobrarán.
- calcularCosto() : double: calcula el costo del servicio utilizando la tarifa base de la zona y agregando Q90 por cada hora facturable.

- calcularTiempoAtencion() : int: devuelve 3 días si la zona es metropolitana y 5 si es periférica.
- maximoVisitas() : int: devuelve 3 porque las instalaciones normales permiten hasta tres visitas.
- getCategoria() : String: devuelve el texto "Instalacion".
-

Clase InstalacionElectrica

- InstalacionElectrica(String codigo, String cliente, String direccion, double horasEstimadas, String zona, int cantidadPuntos, double valorMateriales): constructor que utiliza super() para inicializar los datos heredados de Instalacion. También recibe y valida el valor declarado de los materiales.
- getValorMateriales() : double: devuelve el valor declarado de los materiales utilizados.
- calcularCosto() : double: utiliza super.calcularCosto() para obtener el costo de una instalación normal. Después agrega Q120 por el certificado y el 5 % del valor de los materiales.
- calcularTiempoAtencion() : int: utiliza el tiempo estimado de una instalación normal y agrega dos días adicionales.
- maximoVisitas() : int: devuelve 2 porque las instalaciones eléctricas certificadas permiten como máximo dos visitas.
- getCategoria() : String: devuelve el texto "Instalacion Electrica Certificada".

Clase MantenimientoAire

- MantenimientoAire(String codigo, String cliente, String direccion, double horasEstimadas, String zona, int capacidadBTU): constructor que recibe los datos comunes mediante super() y guarda la capacidad del equipo. Valida que los BTU estén entre 9,000 y 60,000 y que la zona sea metropolitana.
- getCapacidadBTU() : int: devuelve la capacidad del equipo de aire acondicionado.
- calcularCosto() : double: calcula el costo del mantenimiento utilizando la tarifa base correspondiente, Q110 por cada hora estimada y Q80 por el gas refrigerante. La tarifa base específica del mantenimiento no aparece cuantificada de manera independiente en el enunciado, por lo que deberá quedar definida como una decisión de diseño.
- calcularTiempoAtencion() : int: devuelve 2 días porque los mantenimientos de aire acondicionado siempre tienen ese tiempo estimado.
- maximoVisitas() : int: devuelve 1 porque solamente se permite una visita.
- getCategoria() : String: devuelve el texto "Mantenimiento Aire".
- esCompatible(Cuadrilla cuadrilla) : boolean: verifica que la cuadrilla esté certificada en refrigeración. Devuelve true cuando cumple esta condición y false cuando no.

Clase Cuadrilla

- Cuadrilla(String codigo, double capacidadHoras, boolean certificadaRefrigeracion): constructor que recibe el código, la capacidad máxima de trabajo y la certificación. Valida que el código no esté vacío y que la capacidad sea mayor que cero.
- getCodigo() : String: devuelve el código de la cuadrilla.
- getCapacidadHoras() : double: devuelve la capacidad máxima de horas de trabajo por jornada.
- getCertificadaRefrigeracion() : boolean: devuelve true si la cuadrilla está certificada en refrigeración y false si no lo está.

- toString() : String: devuelve la información de la cuadrilla para mostrarla en las consultas.

Clase Jornada

- Jornada(String codigo, Cuadrilla cuadrilla): constructor que recibe el código de la jornada y la cuadrilla responsable. Inicializa el estado como Preparación y crea las colecciones vacías de órdenes y resultados.
- getCodigo() : String: devuelve el código que identifica la jornada.
- getCuadrilla() : Cuadrilla: devuelve la cuadrilla responsable.
- getEstado() : String: devuelve el estado actual de la jornada.
- getOrdenes() : ArrayList de OrdenServicio: devuelve la colección de órdenes asignadas.

- calcularHorasAsignadas() : double: recorre las órdenes de la jornada y suma sus horas estimadas. No utiliza las horas facturables, porque la capacidad de una cuadrilla depende del tiempo estimado de trabajo.
- capacidadDisponible() : double: resta las horas asignadas de la capacidad máxima de la cuadrilla y devuelve las horas que todavía están disponibles.
- asignarOrden(OrdenServicio orden) : boolean: recibe una orden y verifica que la jornada esté en preparación, que la orden esté pendiente, que la cuadrilla sea compatible y que exista capacidad suficiente. Si todas las condiciones se cumplen, agrega la orden a la jornada y cambia su estado a Asignada.
- retirarOrden(OrdenServicio orden) : boolean: recibe una orden y verifica que la jornada esté en preparación y que la orden esté asignada a ella. Si se cumplen las condiciones, la retira y devuelve su estado a Pendiente.
- iniciar() : boolean: verifica que la jornada esté en preparación y tenga al menos una orden. Si puede iniciar, cambia su estado a Iniciada y coloca las órdenes en atención, aumentando las visitas realizadas.
- registrarResultado(String codigoOrden, boolean completada) : boolean: busca la orden correspondiente dentro de la jornada y verifica que esté en atención y que todavía no tenga un resultado registrado. Después registra si la visita fue completada o fallida. Cuando todas las órdenes tengan resultado, cambia la jornada a Cerrada.
- toString() : String: devuelve la información de la jornada, incluyendo su código, cuadrilla, estado, horas asignadas y capacidad disponible.

Clase TecniHogar

- TecniHogar(): constructor que crea las colecciones vacías de órdenes, cuadrillas y jornadas. Inicializa los ingresos por registro y por visitas adicionales en cero.
- registrarOrden(OrdenServicio orden) : boolean: recibe una orden y verifica que su código no esté repetido. Si puede registrarla, la agrega a la colección y suma su costo a los ingresos por registro.
- agregarOrdenInicial(OrdenServicio orden) : boolean: registra una orden inicial sin generar ingresos. Se utiliza para cargar las órdenes pendientes que deben existir al comenzar el programa.
- buscarOrden(String codigo) : OrdenServicio: busca una orden mediante su código y devuelve el objeto encontrado o null si no existe.
- getOrdenes() : ArrayList de OrdenServicio: devuelve las órdenes registradas para poder consultarlas.

- registrarCuadrilla(Cuadrilla cuadrilla) : boolean: agrega una cuadrilla si su código no está repetido y sus datos son válidos.
- buscarCuadrilla(String codigo) : Cuadrilla: busca una cuadrilla mediante su código y devuelve el objeto encontrado o null.
- getCuadrillas() : ArrayList de Cuadrilla: devuelve las cuadrillas registradas.
- crearJornada(String codigo, String codigoCuadrilla) : boolean: busca la cuadrilla indicada y verifica que no tenga otra jornada sin cerrar. También comprueba que el código de la nueva jornada no esté repetido. Si las condiciones se cumplen, crea una jornada en preparación.
- buscarJornada(String codigo) : Jornada: busca una jornada mediante su código y devuelve el objeto encontrado o null.
- getJornadas() : ArrayList de Jornada: devuelve las jornadas registradas.

- asignarOrden(String codigoJornada, String codigoOrden) : boolean: busca la jornada y la orden correspondientes y solicita a la jornada realizar la asignación. Devuelve true si se pudo completar.
- retirarOrden(String codigoJornada, String codigoOrden) : boolean: busca la jornada y la orden correspondientes y solicita retirar la orden si la operación es válida.
- iniciarJornada(String codigoJornada) : boolean: busca la jornada y verifica que pueda iniciar. Si se inicia correctamente, registra Q40 por cada visita adicional a la primera. No debe cobrar nuevamente una visita que ya fue registrada.
- registrarResultado(String codigoJornada, String codigoOrden, boolean completada) : boolean: busca la jornada y registra el resultado de la visita de una orden. Si todas las órdenes tienen resultado, la jornada queda cerrada.
- getIngresosRegistro() : double: devuelve los ingresos acumulados por órdenes confirmadas.
- getIngresosVisitas() : double: devuelve los ingresos acumulados por visitas adicionales.
- calcularIngresosTotales() : double: suma los ingresos por registro y por visitas adicionales.
- generarReporte() : String: recorre las órdenes y jornadas para obtener las cantidades por categoría y estado, los ingresos y las jornadas que todavía no están cerradas. Devuelve la información del reporte en una cadena de texto.
- agregarIngresoCategoria(String categoria, double monto) : void: método privado que recibe el nombre de una categoría y una cantidad de dinero. Busca si la categoría ya existe en la colección y suma el monto correspondiente. Si todavía no existe, registra la categoría junto con su primer ingreso. Este método se utiliza al confirmar órdenes y al registrar cargos por visitas adicionales.

Clase Main

- main(String[] args) : void: método principal que crea el objeto TecniHogar, carga las órdenes y cuadrillas iniciales y mantiene el menú funcionando hasta que el usuario seleccione salir.
- cargarDatosIniciales() : void: crea al menos dos órdenes de cada categoría y tres cuadrillas. Las órdenes comienzan pendientes y no generan ingresos.
- mostrarMenu() : void: muestra las opciones disponibles del programa.
- cotizarOrden() : void: solicita la categoría y los datos del servicio, crea temporalmente el objeto correspondiente y muestra el costo y tiempo estimado sin registrarlo ni modificar los ingresos.

- registrarOrden() : void: permite aceptar una cotización y registrar la orden correspondiente. El costo se agrega a los ingresos solamente cuando el registro se confirma.
- registrarCuadrilla() : void: solicita los datos necesarios para crear una cuadrilla y registrarla en la empresa.
- crearJornada() : void: solicita el código de la jornada y la cuadrilla responsable.
- asignarOrden() : void: solicita la jornada y la orden que se desea asignar.
- retirarOrden() : void: solicita la jornada y la orden que se desea retirar.
- iniciarJornada() : void: solicita una jornada y permite iniciarla si cumple las condiciones necesarias.
- registrarResultado() : void: solicita la jornada, la orden y el resultado de la visita.
- consultarOrdenes() : void: muestra las órdenes registradas y sus características.
- consultarCuadrillas() : void: muestra las cuadrillas registradas.
- mostrarReporte() : void: solicita el reporte a TecniHogar y muestra sus resultados.
- mostrarOrden(OrdenServicio orden) : void: recibe una orden y muestra sus datos, costo, tiempo estimado y estado.
- crearOrden() : OrdenServicio: método privado y estático que solicita al usuario la categoría del servicio y sus datos generales y particulares. Después crea un objeto de la clase correspondiente y lo devuelve como OrdenServicio. Este método no registra órdenes ni modifica ingresos, por lo que puede utilizarse tanto para cotizaciones como para registros confirmados.
- mostrarCuadrilla(Cuadrilla cuadrilla) : void: recibe una cuadrilla y muestra su información.

Validaciones y manejo de operaciones incorrectas

Antes de registrar una orden se verificará que su código no esté vacío ni repetido, que las horas estimadas sean positivas y que la zona sea válida. Cada clase hija realizará sus validaciones particulares. Por ejemplo, Diagnostico rechazará más de 1.5 horas y MantenimientoAire rechazará capacidades fuera del rango permitido o servicios en zona periférica. Para las jornadas se verificará la capacidad, la compatibilidad de la cuadrilla, el estado de la orden y el estado de la jornada antes de realizar cambios. Las operaciones inválidas deberán mostrar un mensaje claro sin modificar la información anterior. También se controlarán las entradas con formato incorrecto para que el programa continúe funcionando. Los ingresos solamente se modificarán cuando se confirme el registro de una orden o cuando comience una jornada que incluya visitas adicionales.

Justificación de la herencia entre Instalacion e InstalacionElectrica

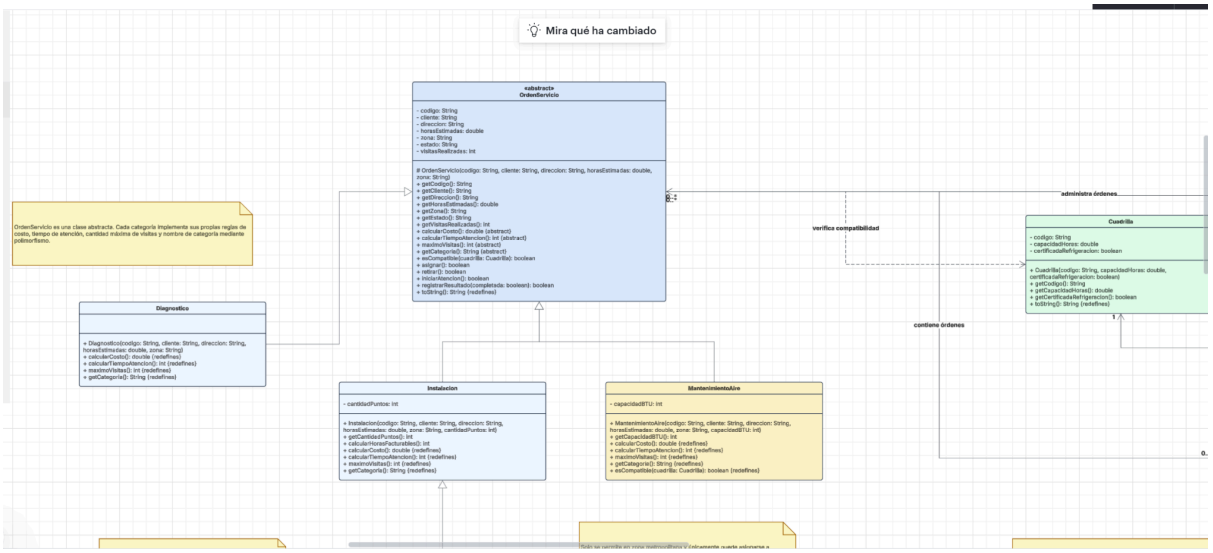
La clase InstalacionElectrica heredará de Instalacion porque ambas comparten varias características y reglas de funcionamiento. Las dos necesitan guardar la cantidad de puntos a instalar y calcular las horas facturables utilizando el mayor valor entre las horas estimadas y las horas calculadas por puntos. Además, las dos utilizan la misma tarifa base según la zona y cobran Q90 por cada hora facturable o fracción. La diferencia es que una instalación eléctrica certificada necesita agregar Q120 por el certificado y el 5 % del valor declarado de los materiales por la garantía. También necesita dos días adicionales para su atención y permite menos visitas que una instalación normal. Por esta razón, InstalacionElectrica puede aprovechar los métodos que ya existen en Instalacion, utilizando super.calcularCosto() para obtener el costo normal y después agregar los cargos

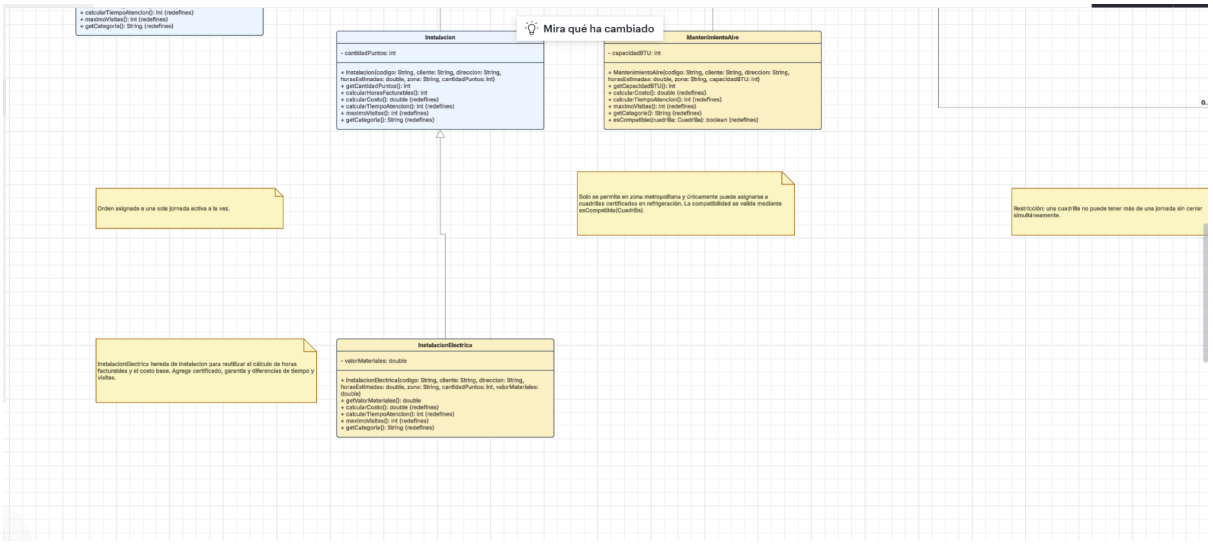
adicionales. Otra opción sería que InstalacionElectrica heredara directamente de OrdenServicio. Sin embargo, esto obligaría a repetir el atributo de cantidad de puntos y los cálculos de horas facturables y costo de instalación. La ventaja de nuestra propuesta es que permite reutilizar código y mantener las responsabilidades mejor organizadas. Si en el futuro cambia el cálculo general de una instalación, ambas categorías podrían aprovechar ese cambio sin tener que modificar el mismo cálculo en dos clases diferentes.

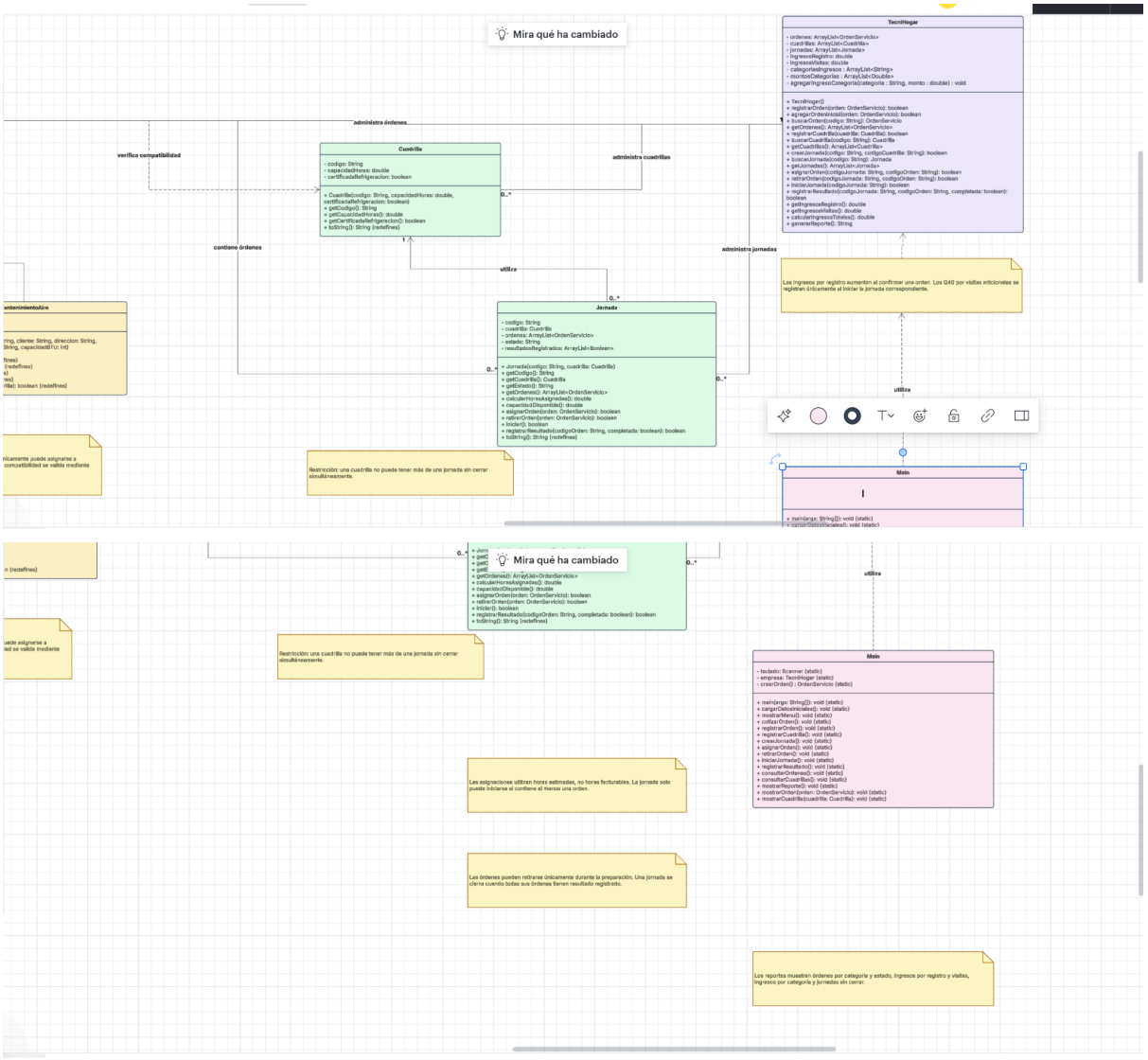
Incorporación de una nueva categoría

Si TecniHogar necesita agregar una nueva categoría de servicio, se podría crear otra clase que herede de OrdenServicio. Por ejemplo, si la empresa incorpora servicios de instalación de paneles solares, se podría crear una clase InstalacionSolar que herede de Instalacion, siempre que comparta las reglas básicas de puntos y horas facturables. La nueva clase agregaría solamente sus atributos particulares y sobrescribiría los métodos necesarios para calcular su costo, tiempo de atención, compatibilidad con cuadrillas y cantidad máxima de visitas. Gracias al polimorfismo, las clases TecniHogar y Jornada podrían trabajar con los objetos de la nueva categoría mediante los métodos comunes de OrdenServicio, sin tener que preguntar constantemente qué tipo de servicio se está utilizando. La clase Jornada no necesitaría cambiar sus reglas generales de asignación, porque la compatibilidad con la cuadrilla y las visitas permitidas se consultarían mediante los métodos de cada categoría. Los reportes también podrían utilizar getCategoria() para identificar la nueva categoría sin utilizar instanceof. Sin embargo, si el reporte tiene nombres de categorías escritos directamente, habría que actualizar esa parte para incluir la nueva categoría. También sería necesario modificar el menú de Main para permitir que el usuario seleccione y registre servicios de la nueva categoría. De esta manera, el sistema podría incorporar nuevos servicios sin tener que modificar los cálculos de las categorías que ya existen.

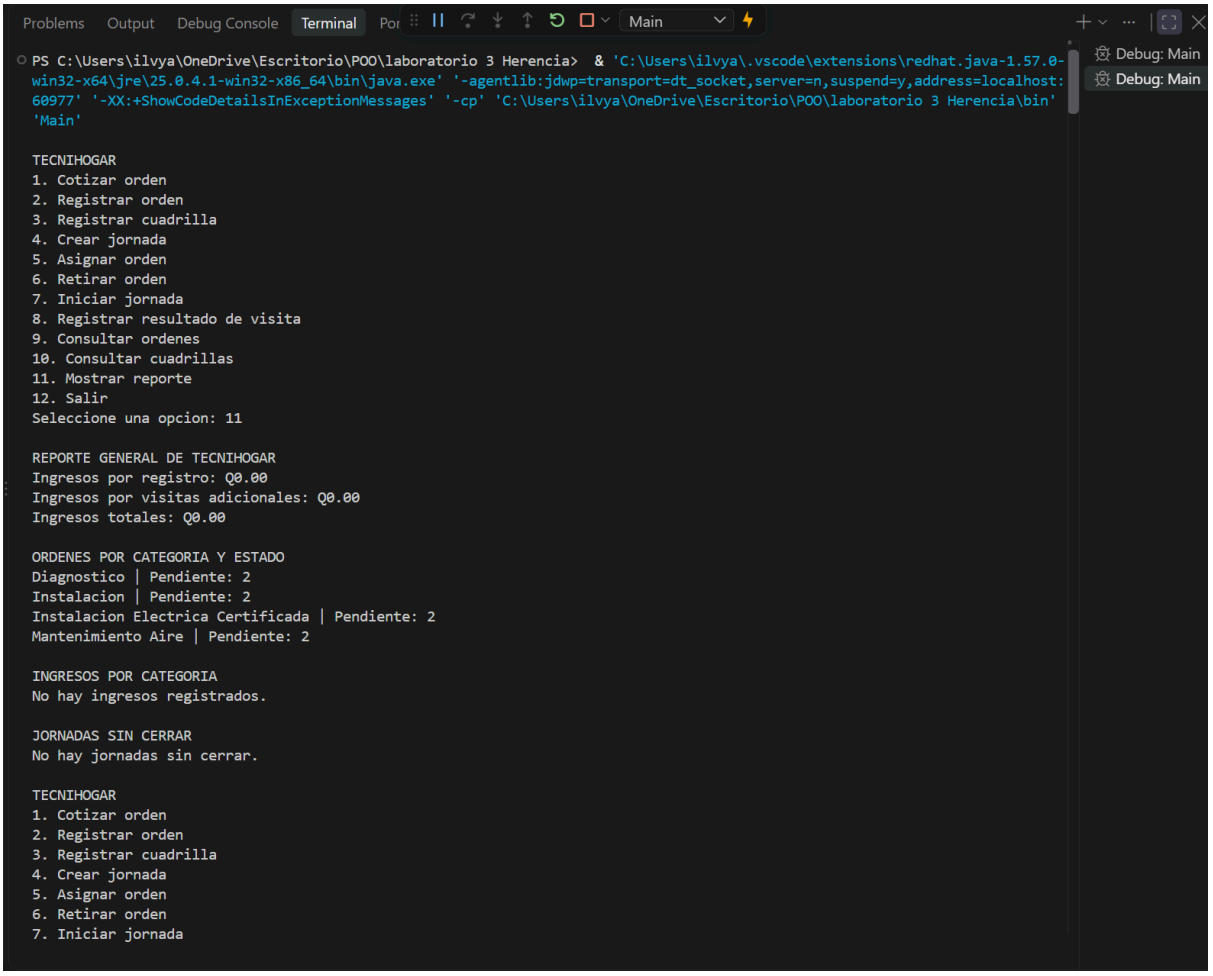
2. UML

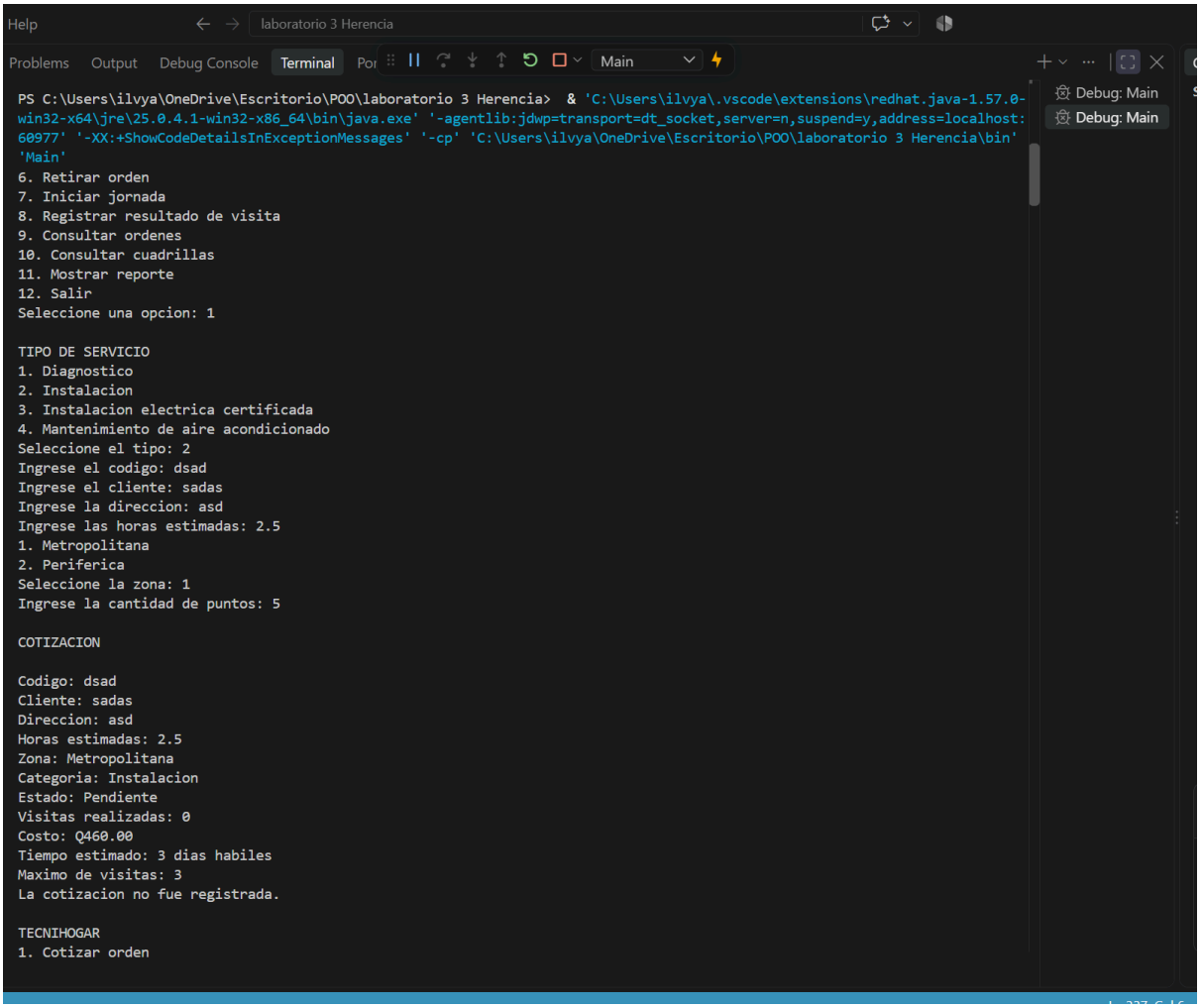

Mira qué ha cambiado
«abstract»
OrdenServicio
- codigo: String
- cliente: String
- direccion: String
- horasEstimadas: double
- zona: String
- estado: String
- visitasRealizadas: int
OrdenServicio(codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: String)
+ getCodigo(): String
+ getCliente(): String
+ getDireccion(): String
+ getHorasEstimadas(): double
+ getZona(): String
+ getEstado(): String
+ getVisitasRealizadas(): int
+ calcularCosto(): double {abstract}
+ calcularTiempoAtencion(): Int {abstract}
+ maximoVisitas(): Int {abstract}
+ getCategoria(): String {abstract}
+ esCompatible(cuadrilla: Cuadrilla): boolean
+ asignar(): boolean
+ retirar(): boolean
+ iniciarAtencion(): boolean
+ registrarResultado(completada: boolean): boolean
+ toString(): String {redefines}
OrdenServicio es una clase abstracta. Cada categoría implementa sus propias reglas de costo, tiempo de atención, cantidad máxima de visitas y nombre de categoría mediante polimorfismo.
administra órdenes
verifica compatibilidad
Cuadrilla
- codigo: String
- capacidadHoras: double
- certificadaRefrigeracion: boolean
+ Cuadrilla(codigo: String, capacidadHoras: double, certificadaRefrigeracion: boolean)
+ getCodigo(): String
+ getCapacidadHoras(): double
+ getCertificadaRefrigeracion(): boolean
+ toString(): String {redefines}
1
contiene órdenes
0..
Diagnostico
+ Diagnostico(codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: String)
+ calcularCosto(): double {redefines}
+ calcularTiempoAtencion(): Int {redefines}
+ maximoVisitas(): Int {redefines}
+ getCategoria(): String {redefines}
Instalacion
- cantidadPuntos: int
+ Instalacion(codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: String, cantidadPuntos: Int)
+ getCantidadPuntos(): Int
+ calcularHorasFacturables(): Int
+ calcularCosto(): double {redefines}
+ calcularTiempoAtencion(): Int {redefines}
+ maximoVisitas(): Int {redefines}
+ getCategoria(): String {redefines}
MantenimientoAire
- capacidadBTU: Int
+ MantenimientoAire(codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: String, capacidadBTU: Int)
+ getCapacidadBTU(): Int
+ calcularCosto(): double {redefines}
+ calcularTiempoAtencion(): Int {redefines}
+ maximoVisitas(): Int {redefines}
+ getCategoria(): String {redefines}
+ esCompatible(cuadrilla: Cuadrilla): boolean {redefines}

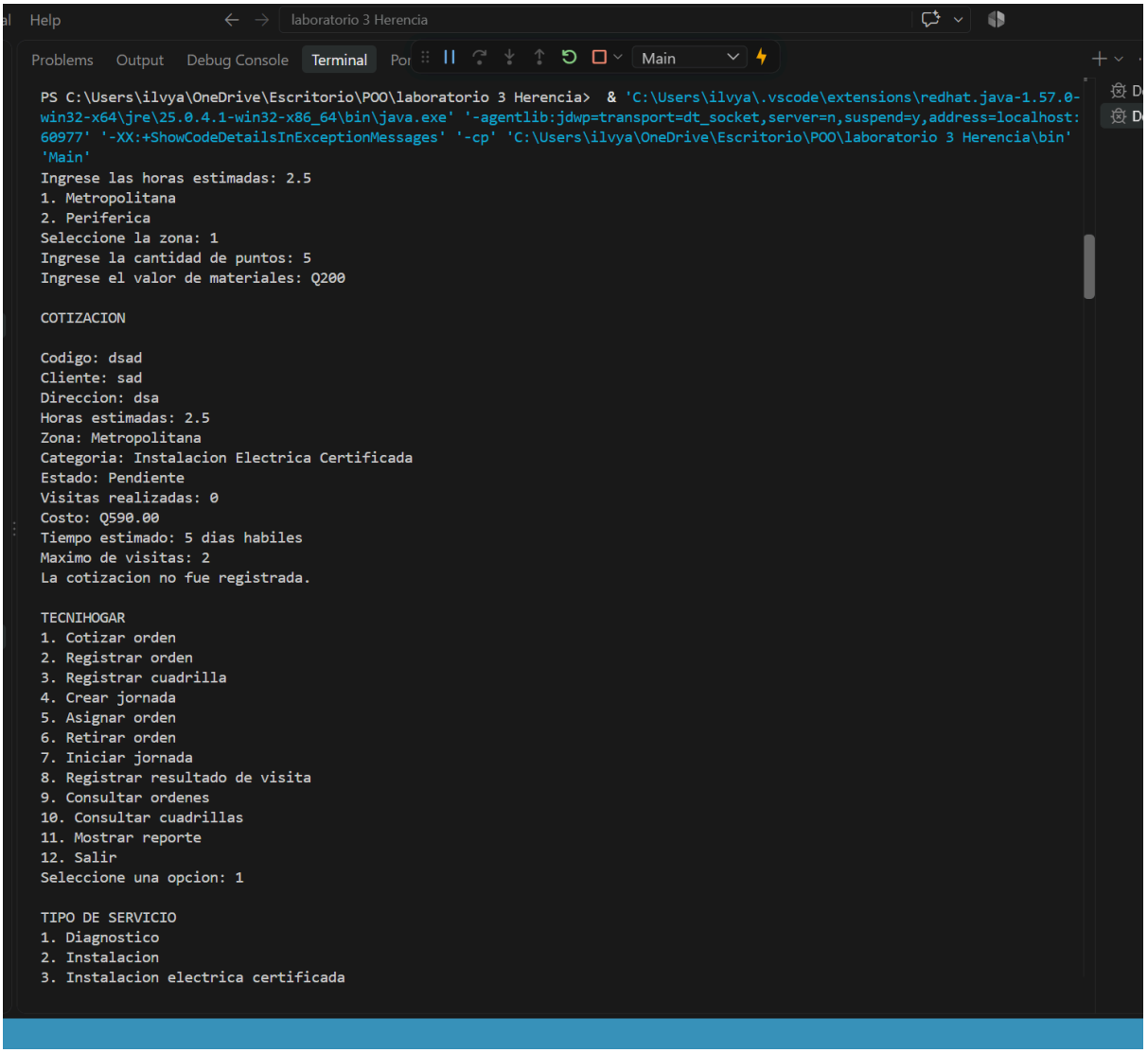

Mira qué ha cambiado
+ calcularTiempoAtencion(): Int {redefines}
+ maximoVisitas(): Int {redefines}
+ getCategoria(): String {redefines}
Instalacion
- cantidadPuntos: int
+ Instalacion(codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: String, cantidadPuntos: Int)
+ getCantidadPuntos(): Int
+ calcularHorasFacturables(): Int
+ calcularCosto(): double {redefines}
+ calcularTiempoAtencion(): Int {redefines}
+ maximoVisitas(): Int {redefines}
+ getCategoria(): String {redefines}
MantenimientoAire
- capacidadBTU: Int
+ MantenimientoAire(codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: String, capacidadBTU: Int)
+ getCapacidadBTU(): Int
+ calcularCosto(): double {redefines}
+ calcularTiempoAtencion(): Int {redefines}
+ maximoVisitas(): Int {redefines}
+ getCategoria(): String {redefines}
+ esCompatible(cuadrilla: Cuadrilla): boolean {redefines}
0..
Orden asignada a una sola jornada activa a la vez.
Solo se permite en zona metropolitana y únicamente puede asignarse a cuadrillas certificadas en refrigeración. La compatibilidad se valida mediante esCompatible(Cuadrilla).
Restricción: una cuadrilla no puede tener más de una jornada sin cerrar simultáneamente.
InstalacionElectrica
- valorMateriales: double
+ InstalacionElectrica(codigo: String, cliente: String, direccion: String, horasEstimadas: double, zona: String, cantidadPuntos: Int, valorMateriales: double)
+ getValorMateriales(): double
+ calcularCosto(): double {redefines}
+ calcularTiempoAtencion(): Int {redefines}
+ maximoVisitas(): Int {redefines}
+ getCategoria(): String {redefines}
InstalacionElectrica hereda de Instalacion para reutilizar el cálculo de horas facturables y el costo base. Agrega certificado, garantía y diferencias de tiempo y visitas.

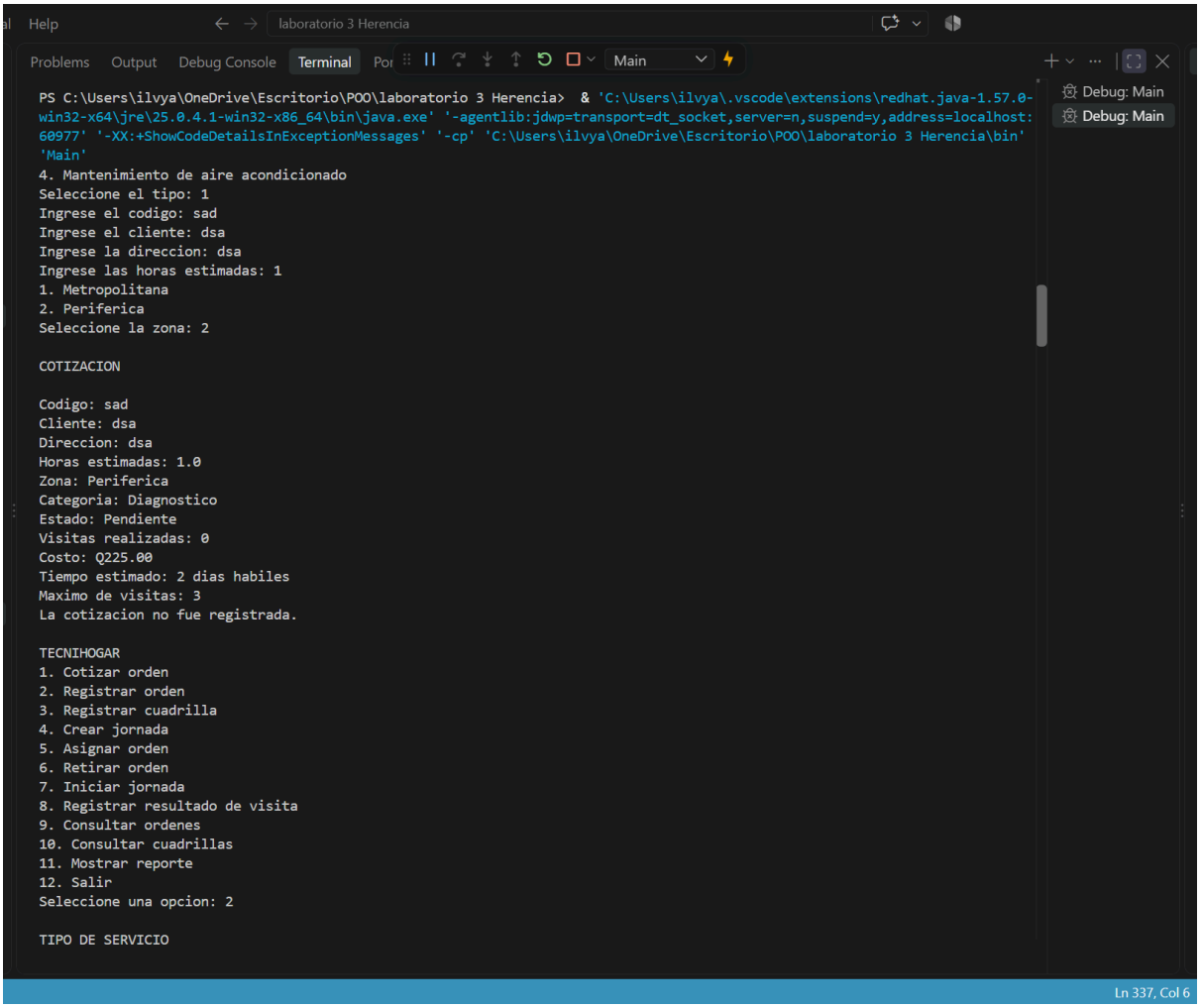

Mira qué ha cambiado
TecniHogar
administra órdenes
verifica compatibilidad
Cuadrilla
administra cuadrillas
contiene órdenes
utiliza
administra jornadas
Jornada
Los ingresos por registro aumentan al confirmar una orden. Los Q40 por visitas aficionales se registran únicamente al iniciar la jornada correspondiente.
Restricción: una cuadrilla no puede tener más de una jornada sin cerrar simultáneamente.
Main
Mira qué ha cambiado
Restricción: una cuadrilla no puede tener más de una jornada sin cerrar simultáneamente.
Las asignaciones utilizan horas estimadas, no horas facturables. La jornada solo puede iniciarse si contiene al menos una orden.
Las órdenes pueden retirarse únicamente durante la preparación. Una jornada se cierra cuando todas sus órdenes tienen resultado registrado.
Los reportes muestran órdenes por categoría y estado, ingresos por registro y visitas, ingresos por categoría y jornadas sin cerrar.

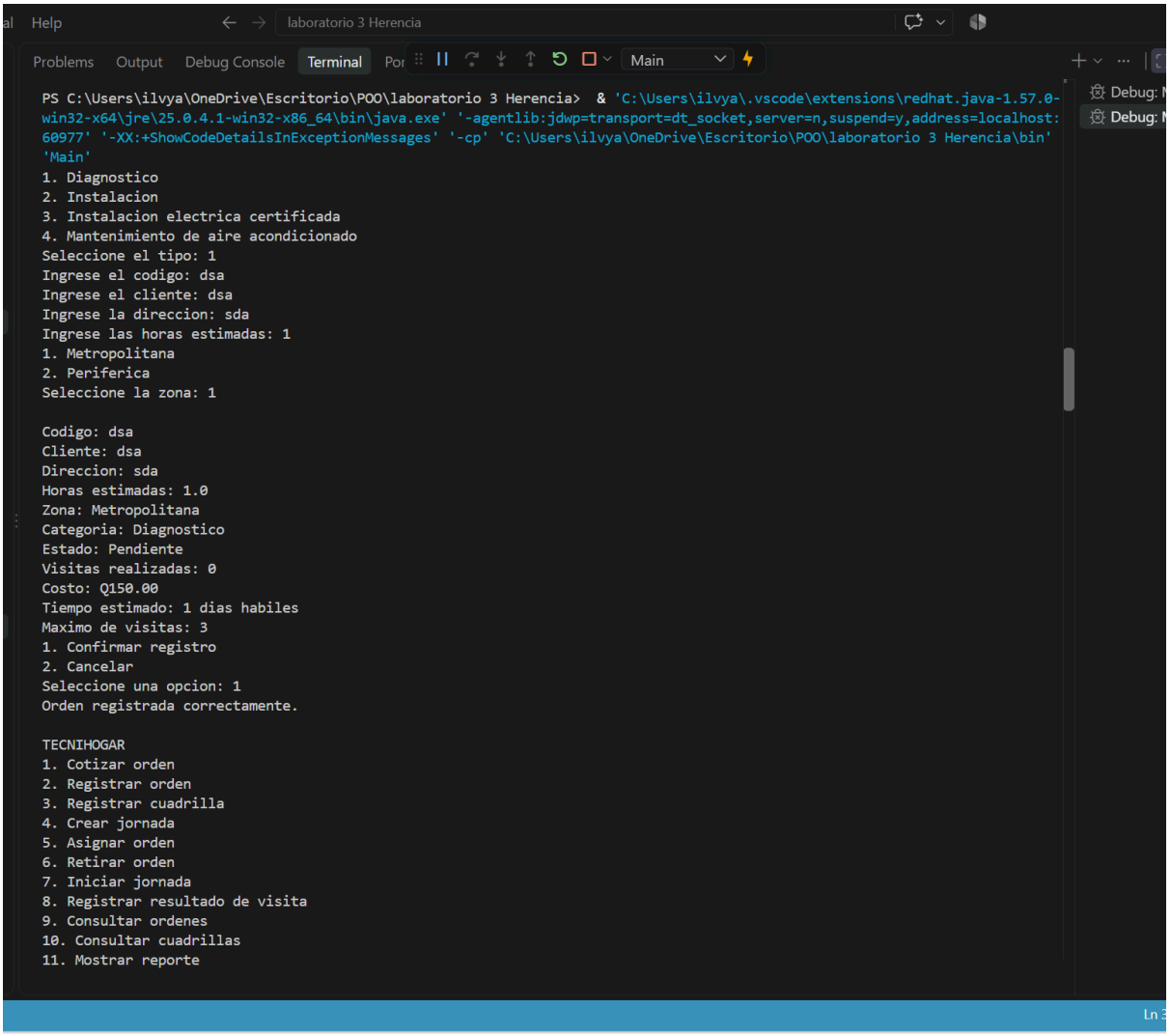
3. Pruebas

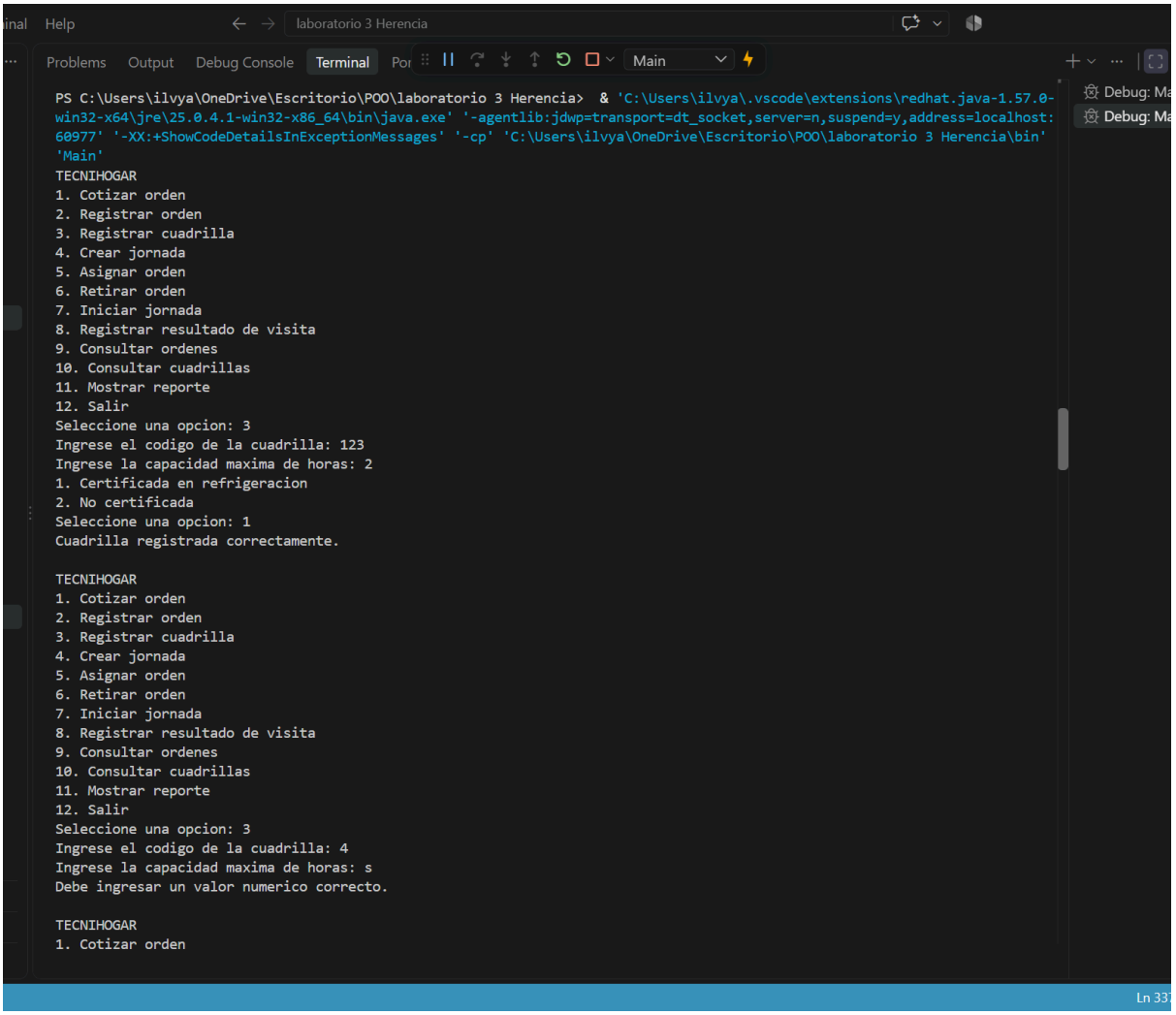

Problems
Output
Debug Console
Terminal
Por
Main
Debug: Main
Debug: Main
PS C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia> & 'C:\Users\ilvya\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:60977' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia\bin' 'Main'
TECNIHOGAR
1. Cotizar orden
2. Registrar orden
3. Registrar cuadrilla
4. Crear jornada
5. Asignar orden
6. Retirar orden
7. Iniciar jornada
8. Registrar resultado de visita
9. Consultar ordenes
10. Consultar cuadrillas
11. Mostrar reporte
12. Salir
Seleccione una opcion: 11
REPORTE GENERAL DE TECNIHOGAR
Ingresos por registro: Q0.00
Ingresos por visitas adicionales: Q0.00
Ingresos totales: Q0.00
ORDENES POR CATEGORIA Y ESTADO
Diagnostico | Pendiente: 2
Instalacion | Pendiente: 2
Instalacion Electrica Certificada | Pendiente: 2
Mantenimiento Aire | Pendiente: 2
INGRESOS POR CATEGORIA
No hay ingresos registrados.
JORNADAS SIN CERRAR
No hay jornadas sin cerrar.
TECNIHOGAR
1. Cotizar orden
2. Registrar orden
3. Registrar cuadrilla
4. Crear jornada
5. Asignar orden
6. Retirar orden
7. Iniciar jornada

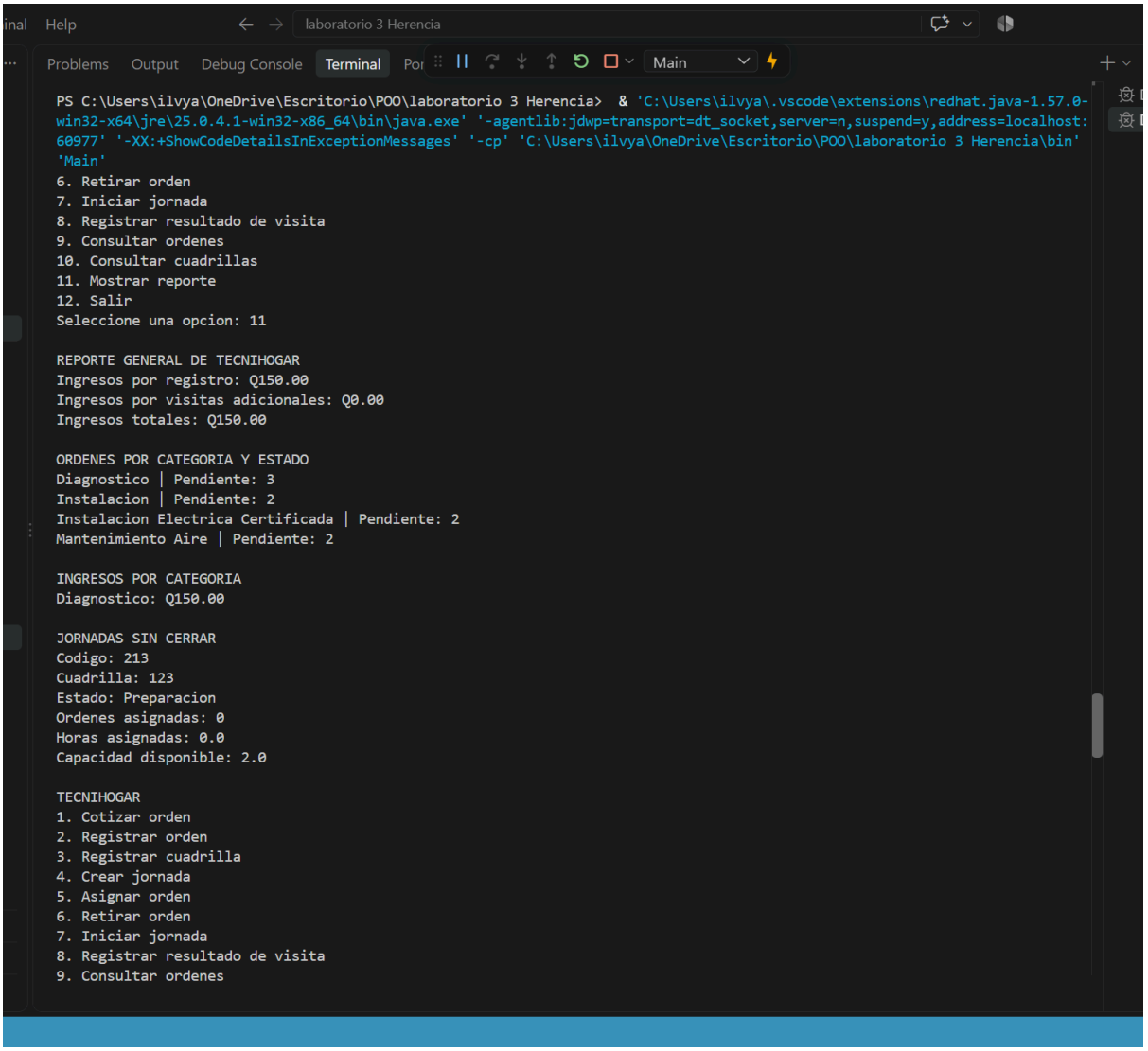

Help
laboratorio 3 Herencia
Problems
Output
Debug Console
Terminal
Main
Debug: Main
Debug: Main
PS C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia> & 'C:\Users\ilvya\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:60977' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia\bin' 'Main'
6. Retirar orden
7. Iniciar jornada
8. Registrar resultado de visita
9. Consultar ordenes
10. Consultar cuadrillas
11. Mostrar reporte
12. Salir
Seleccione una opcion: 1
TIPO DE SERVICIO
1. Diagnostico
2. Instalacion
3. Instalacion electrica certificada
4. Mantenimiento de aire acondicionado
Seleccione el tipo: 2
Ingrese el codigo: dsad
Ingrese el cliente: sadas
Ingrese la direccion: asd
Ingrese las horas estimadas: 2.5
1. Metropolitana
2. Periferica
Seleccione la zona: 1
Ingrese la cantidad de puntos: 5
COTIZACION
Codigo: dsad
Cliente: sadas
Direccion: asd
Horas estimadas: 2.5
Zona: Metropolitana
Categoria: Instalacion
Estado: Pendiente
Visitas realizadas: 0
Costo: Q460.00
Tiempo estimado: 3 dias habiles
Maximo de visitas: 3
La cotizacion no fue registrada.
TECNIHOGAR
1. Cotizar orden

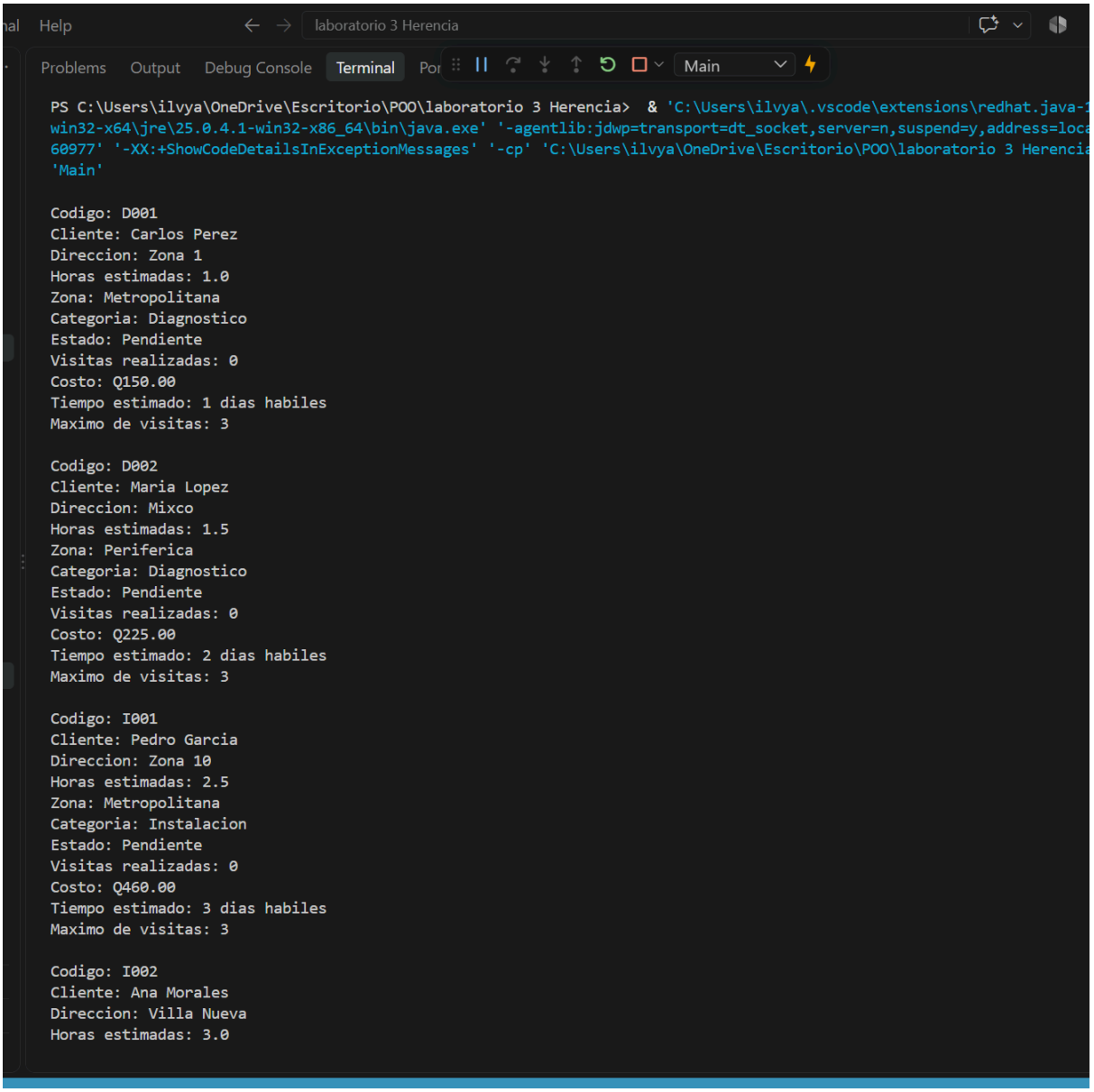

Help
laboratorio 3 Herencia
Problems
Output
Debug Console
Terminal
Main
PS C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia> & 'C:\Users\ilvya\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:60977' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia\bin' 'Main'
Ingrese las horas estimadas: 2.5
1. Metropolitana
2. Periferica
Seleccione la zona: 1
Ingrese la cantidad de puntos: 5
Ingrese el valor de materiales: Q200
COTIZACION
Codigo: dsad
Cliente: sad
Direccion: dsa
Horas estimadas: 2.5
Zona: Metropolitana
Categoria: Instalacion Electrica Certificada
Estado: Pendiente
Visitas realizadas: 0
Costo: Q590.00
Tiempo estimado: 5 dias habiles
Maximo de visitas: 2
La cotizacion no fue registrada.
TECNIHOGAR
1. Cotizar orden
2. Registrar orden
3. Registrar cuadrilla
4. Crear jornada
5. Asignar orden
6. Retirar orden
7. Iniciar jornada
8. Registrar resultado de visita
9. Consultar ordenes
10. Consultar cuadrillas
11. Mostrar reporte
12. Salir
Seleccione una opcion: 1
TIPO DE SERVICIO
1. Diagnostico
2. Instalacion
3. Instalacion electrica certificada

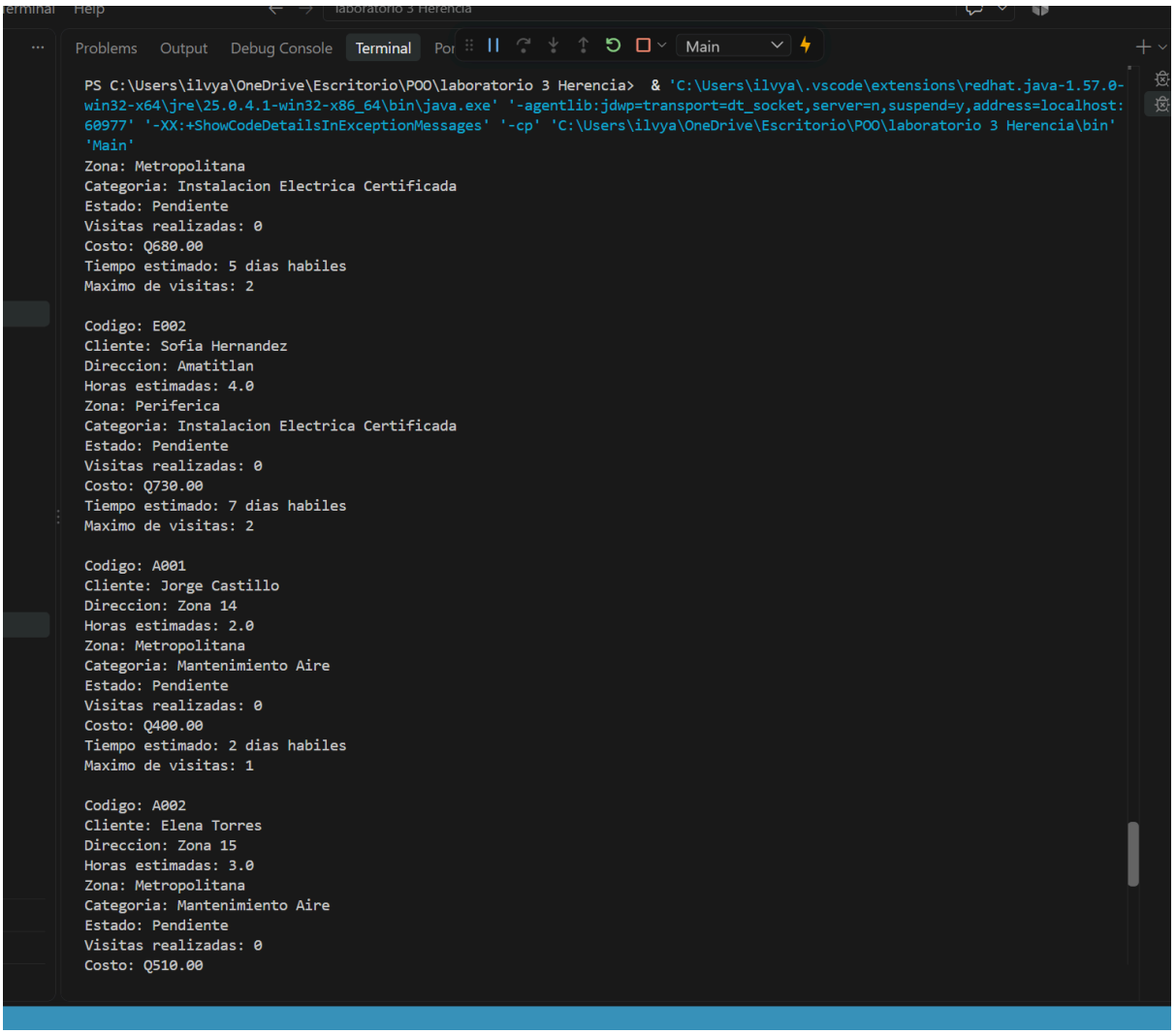

Help
laboratorio 3 Herencia
Problems
Output
Debug Console
Terminal
Main
Debug: Main
Debug: Main
PS C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia> & 'C:\Users\ilvya\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:60977' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia\bin' 'Main'
4. Mantenimiento de aire acondicionado
Seleccione el tipo: 1
Ingrese el codigo: sad
Ingrese el cliente: dsa
Ingrese la direccion: dsa
Ingrese las horas estimadas: 1
1. Metropolitana
2. Periferica
Seleccione la zona: 2
COTIZACION
Codigo: sad
Cliente: dsa
Direccion: dsa
Horas estimadas: 1.0
Zona: Periferica
Categoria: Diagnostico
Estado: Pendiente
Visitas realizadas: 0
Costo: Q225.00
Tiempo estimado: 2 dias habiles
Maximo de visitas: 3
La cotizacion no fue registrada.
TECNIHOGAR
1. Cotizar orden
2. Registrar orden
3. Registrar cuadrilla
4. Crear jornada
5. Asignar orden
6. Retirar orden
7. Iniciar jornada
8. Registrar resultado de visita
9. Consultar ordenes
10. Consultar cuadrillas
11. Mostrar reporte
12. Salir
Seleccione una opcion: 2
TIPO DE SERVICIO
Ln 337, Col 6

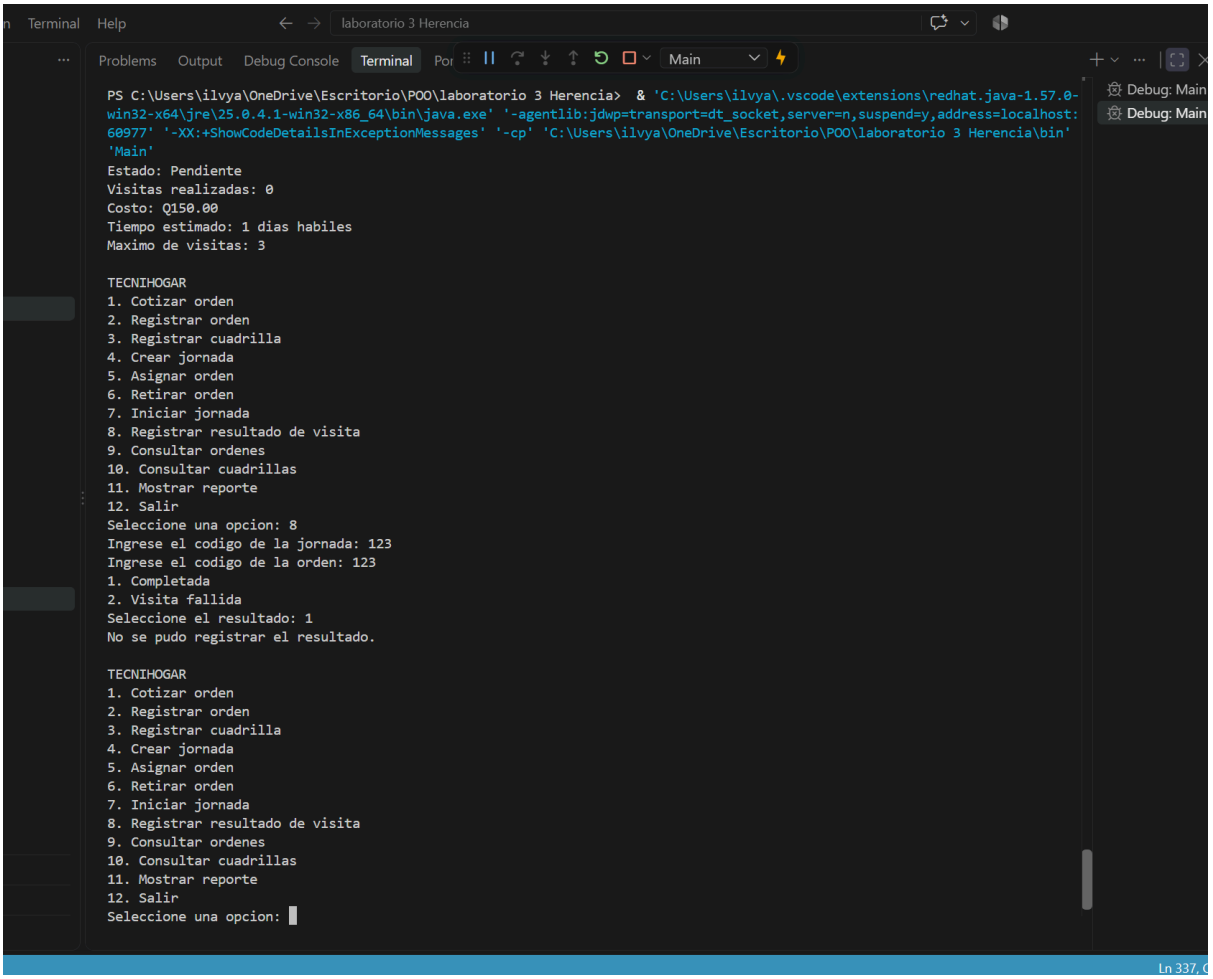

Help
laboratorio 3 Herencia
Problems
Output
Debug Console
Terminal
Main
PS C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia> & 'C:\Users\ilvya\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:60977' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia\bin' 'Main'
1. Diagnostico
2. Instalacion
3. Instalacion electrica certificada
4. Mantenimiento de aire acondicionado
Seleccione el tipo: 1
Ingrese el codigo: dsa
Ingrese el cliente: dsa
Ingrese la direccion: sda
Ingrese las horas estimadas: 1
1. Metropolitana
2. Periferica
Seleccione la zona: 1
Codigo: dsa
Cliente: dsa
Direccion: sda
Horas estimadas: 1.0
Zona: Metropolitana
Categoria: Diagnostico
Estado: Pendiente
Visitas realizadas: 0
Costo: Q150.00
Tiempo estimado: 1 dias habiles
Maximo de visitas: 3
1. Confirmar registro
2. Cancelar
Seleccione una opcion: 1
Orden registrada correctamente.
TECNIHOGAR
1. Cotizar orden
2. Registrar orden
3. Registrar cuadrilla
4. Crear jornada
5. Asignar orden
6. Retirar orden
7. Iniciar jornada
8. Registrar resultado de visita
9. Consultar ordenes
10. Consultar cuadrillas
11. Mostrar reporte
Debug:
Debug:

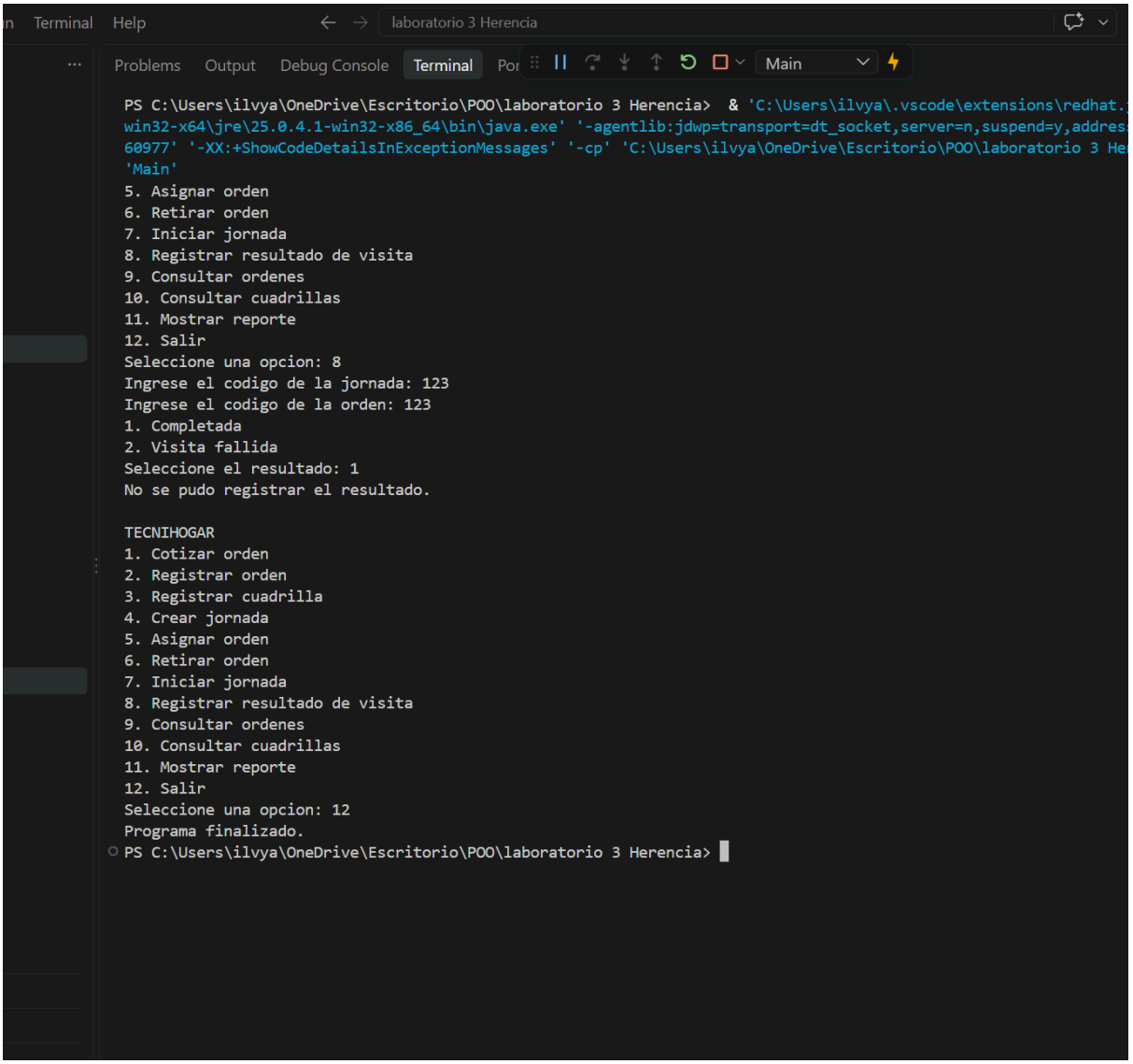

Help
laboratorio 3 Herencia
Problems
Output
Debug Console
Terminal
Main
Debug: Ma
Debug: Ma
PS C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia> & 'C:\Users\ilvya\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:60977' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia\bin' 'Main'
TECNIHOGAR
1. Cotizar orden
2. Registrar orden
3. Registrar cuadrilla
4. Crear jornada
5. Asignar orden
6. Retirar orden
7. Iniciar jornada
8. Registrar resultado de visita
9. Consultar ordenes
10. Consultar cuadrillas
11. Mostrar reporte
12. Salir
Seleccione una opcion: 3
Ingrese el codigo de la cuadrilla: 123
Ingrese la capacidad maxima de horas: 2
1. Certificada en refrigeracion
2. No certificada
Seleccione una opcion: 1
Cuadrilla registrada correctamente.
TECNIHOGAR
1. Cotizar orden
2. Registrar orden
3. Registrar cuadrilla
4. Crear jornada
5. Asignar orden
6. Retirar orden
7. Iniciar jornada
8. Registrar resultado de visita
9. Consultar ordenes
10. Consultar cuadrillas
11. Mostrar reporte
12. Salir
Seleccione una opcion: 3
Ingrese el codigo de la cuadrilla: 4
Ingrese la capacidad maxima de horas: s
Debe ingresar un valor numerico correcto.
TECNIHOGAR
1. Cotizar orden


inal Help
laboratorio 3 Herencia
Problems Output Debug Console Terminal Por
Main
PS C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia> & 'C:\Users\ilvya\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:60977' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia\bin' 'Main'
6. Retirar orden
7. Iniciar jornada
8. Registrar resultado de visita
9. Consultar ordenes
10. Consultar cuadrillas
11. Mostrar reporte
12. Salir
Seleccione una opcion: 11
REPORTE GENERAL DE TECNIHOGAR
Ingresos por registro: Q150.00
Ingresos por visitas adicionales: Q0.00
Ingresos totales: Q150.00
ORDENES POR CATEGORIA Y ESTADO
Diagnostico | Pendiente: 3
Instalacion | Pendiente: 2
Instalacion Electrica Certificada | Pendiente: 2
Mantenimiento Aire | Pendiente: 2
INGRESOS POR CATEGORIA
Diagnostico: Q150.00
JORNADAS SIN CERRAR
Codigo: 213
Cuadrilla: 123
Estado: Preparacion
Ordenes asignadas: 0
Horas asignadas: 0.0
Capacidad disponible: 2.0
TECNIHOGAR
1. Cotizar orden
2. Registrar orden
3. Registrar cuadrilla
4. Crear jornada
5. Asignar orden
6. Retirar orden
7. Iniciar jornada
8. Registrar resultado de visita
9. Consultar ordenes


Help
laboratorio 3 Herencia
Problems
Output
Debug Console
Terminal
Main
PS C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia> & 'C:\Users\ilvya\.vscode\extensions\redhat.java-1
win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=loca
60977' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia
'Main'
Codigo: D001
Cliente: Carlos Perez
Direccion: Zona 1
Horas estimadas: 1.0
Zona: Metropolitana
Categoria: Diagnostico
Estado: Pendiente
Visitas realizadas: 0
Costo: Q150.00
Tiempo estimado: 1 dias habiles
Maximo de visitas: 3
Codigo: D002
Cliente: Maria Lopez
Direccion: Mixco
Horas estimadas: 1.5
Zona: Periferica
Categoria: Diagnostico
Estado: Pendiente
Visitas realizadas: 0
Costo: Q225.00
Tiempo estimado: 2 dias habiles
Maximo de visitas: 3
Codigo: I001
Cliente: Pedro Garcia
Direccion: Zona 10
Horas estimadas: 2.5
Zona: Metropolitana
Categoria: Instalacion
Estado: Pendiente
Visitas realizadas: 0
Costo: Q460.00
Tiempo estimado: 3 dias habiles
Maximo de visitas: 3
Codigo: I002
Cliente: Ana Morales
Direccion: Villa Nueva
Horas estimadas: 3.0


Terminal Help
laboratorio 3 Herencia
Problems
Output
Debug Console
Terminal
Main
PS C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia> & 'C:\Users\ilvya\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:60977' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia\bin' 'Main'
Zona: Metropolitana
Categoria: Instalacion Electrica Certificada
Estado: Pendiente
Visitas realizadas: 0
Costo: Q680.00
Tiempo estimado: 5 dias habiles
Maximo de visitas: 2
Codigo: E002
Cliente: Sofia Hernandez
Direccion: Amatitlan
Horas estimadas: 4.0
Zona: Periferica
Categoria: Instalacion Electrica Certificada
Estado: Pendiente
Visitas realizadas: 0
Costo: Q730.00
Tiempo estimado: 7 dias habiles
Maximo de visitas: 2
Codigo: A001
Cliente: Jorge Castillo
Direccion: Zona 14
Horas estimadas: 2.0
Zona: Metropolitana
Categoria: Mantenimiento Aire
Estado: Pendiente
Visitas realizadas: 0
Costo: Q400.00
Tiempo estimado: 2 dias habiles
Maximo de visitas: 1
Codigo: A002
Cliente: Elena Torres
Direccion: Zona 15
Horas estimadas: 3.0
Zona: Metropolitana
Categoria: Mantenimiento Aire
Estado: Pendiente
Visitas realizadas: 0
Costo: Q510.00


Terminal Help
laboratorio 3 Herencia
Problems Output Debug Console Terminal Por
Main
Debug: Main
Debug: Main
PS C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia> & 'C:\Users\ilvya\.vscode\extensions\redhat.java-1.57.0-win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,address=localhost:60977' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia\bin' 'Main'
Estado: Pendiente
Visitas realizadas: 0
Costo: Q150.00
Tiempo estimado: 1 dias habiles
Maximo de visitas: 3
TECNIHOGAR
1. Cotizar orden
2. Registrar orden
3. Registrar cuadrilla
4. Crear jornada
5. Asignar orden
6. Retirar orden
7. Iniciar jornada
8. Registrar resultado de visita
9. Consultar ordenes
10. Consultar cuadrillas
11. Mostrar reporte
12. Salir
Seleccione una opcion: 8
Ingrese el codigo de la jornada: 123
Ingrese el codigo de la orden: 123
1. Completada
2. Visita fallida
Seleccione el resultado: 1
No se pudo registrar el resultado.
TECNIHOGAR
1. Cotizar orden
2. Registrar orden
3. Registrar cuadrilla
4. Crear jornada
5. Asignar orden
6. Retirar orden
7. Iniciar jornada
8. Registrar resultado de visita
9. Consultar ordenes
10. Consultar cuadrillas
11. Mostrar reporte
12. Salir
Seleccione una opcion:
Ln 337,


Terminal Help
laboratorio 3 Herencia
Problems Output Debug Console Terminal Por
Main
PS C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia> & 'C:\Users\ilvya\.vscode\extensions\redhat.
win32-x64\jre\25.0.4.1-win32-x86_64\bin\java.exe' '-agentlib:jdwp=transport=dt_socket,server=n,suspend=y,addres
60977' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 He
'Main'
5. Asignar orden
6. Retirar orden
7. Iniciar jornada
8. Registrar resultado de visita
9. Consultar ordenes
10. Consultar cuadrillas
11. Mostrar reporte
12. Salir
Seleccione una opcion: 8
Ingrese el codigo de la jornada: 123
Ingrese el codigo de la orden: 123
1. Completada
2. Visita fallida
Seleccione el resultado: 1
No se pudo registrar el resultado.
TECNIHOGAR
1. Cotizar orden
2. Registrar orden
3. Registrar cuadrilla
4. Crear jornada
5. Asignar orden
6. Retirar orden
7. Iniciar jornada
8. Registrar resultado de visita
9. Consultar ordenes
10. Consultar cuadrillas
11. Mostrar reporte
12. Salir
Seleccione una opcion: 12
Programa finalizado.
PS C:\Users\ilvya\OneDrive\Escritorio\POO\laboratorio 3 Herencia>